London South Bank University

EST 1892

School of Engineering

INITIAL PROGRESS REPORT

AI Customer Assistant Chatbot with Integrated Ticketing

Student ID : 4127677

Lecturer : Mike Child

Supervisor: Dr.Daqing Cheng

Course : BSc. Computer Science

Submit Date:14/12/2024

Project Title: AI Customer Assistant Chatbot with Integrated Ticketing

Project Supervisor: Dr. Daqing Chen	**Contact Details:** chend@lsbu.ac.uk

Project Objectives (3-5 lines)

Project aim to develop an intelligent AI customer assistant chat bot system that provide real time customer assistance service for the users. The system will,

- Handle real-time customer queries.
- Use integrates ticketing system for unresolved issues, providing context for human agents
- to provide answers.
- continuously improve the chatbot's functionality through machine learning and user
- feedback.
- enhance customer satisfaction and support team efficiency.
- Provide "RESTful API" to integrate the chatbot into various platforms.

Project Description (2-3 paragraphs)

Human-centered customer service was used until the advent of artificial intelligence. Customers were contacted by human agents via phone, email, and in-person interactions. Human agents were also expensive to train and maintain 24/7 availability. For these reasons, people have developed chatbots that respond to specific queries based on predefined keywords and a limited set of decision trees.

Nowadays, many businesses and websites use AI-powered chatbots to manage the queries of their customers. But this project will be developed using "TensorFlow" (for machine learning and natural language processing) and a variety of other tools to help businesses manage customer queries. The import part of this project is the integrated ticketing mechanism. As a result, human agents can pick up where the chatbot left off, minimizing customer frustration. In addition, the AI chatbot will continuously learn from the responses of human agents and will be able to improve its response over time. As the chatbot evolves, this function eliminates the need for human intervention in repetitive queries.

AI chatbots are capable of answering different types of questions for customers and increasing user engagement with businesses and websites. In addition, the project website provides APIs for integrating the chatbot with various platforms. In order to create an AI chat-bot, users need to provide a dataset about their business or website; once it has learned the queries and answered them, it will be able to provide very quick answers to their customers.

Project Output

☐ System Design and evaluation

☐ Comparative overview or study and evaluation framework

- ✓ System Analysis and Modelling

☐ Theoretical analysis, Algorithm Design and Development

Required HW/SW

WHAT	WHERE (e.g. University's premises, Student's PC etc.)
Web server (Node.js environment)	Student's PC and Hosting Services
Database (PostgreSQL)	Hosting Service
TensorFlow/TensorFlow.js	Student's PC for development
AWS S3 Bucket (Storage)	Hosting Service
React/Next.js Framework	Student's PC for frontend development
Tailwind CSS	Student's PC
AWS SageMaker	Cloud Service
Prettier & ESLint	Student's PC

Other Requirements

WHAT	LEVEL	IMPACT
Programming	Intermediate to Advanced	Essential for building the frontend, backend, and integration with APIs.
Algorithm Design	Intermediate	Required for implementing machine learning models and ticket escalation logic.
UI/UX Design	Basic	Ensures user-friendly interface for customers interacting with the chatbot.
API Integration	Intermediate	Critical for integrating external data sources and deploying chatbot features.
Cloud Deployment	Intermediate	Needed for hosting and scaling the application using platforms like AWS or Railway.

This project aligns with academic goals and offers practical industry applications. I am committed to completing the project successfully and look forward to your approval and guidance before proceeding.

Table of Contents

Introduction:.....5
Literature Review: 6
Technical Review: 9
Machine Learning: TensaFlow.....10
Frontend: React/Next.js 10
Backend: Node.js 11
Backend: PostgreSQL 12
Deployment: 13
Frontend: Vercel or Netlify 13
Frontend: Railway 13
Database: PostgreSQL(Neon).....13
Storage: AWS S3 Bucket 13
Machine Learning Model: AWS SageMaker.....14
Architecture of the project.....14
Implementation: 15
Sprint 1 :Design and Architecture.....17
Sprint 2 :Building Frontend 17
Sprint 3 :Backend Development and Integration.....17
Sprint 4 : Chatbot Integration and Core Functionalities.....18
Sprint 5 : Advanced Features , Final Testing and deployment.....18
References: 19

Introduction:

Until the advent of artificial intelligence, customer service was human-cantered. Human agents contacted customers via phone, email, and in-person interactions. It was also costly to train and maintain the 24/7 availability of human agents. For these reasons, people have developed chatbots that respond to specific queries based on predefined keywords and a limited set of decision trees. In 1990, MIT professor "Joseph Weiznbanm" developed the first chatbot. Early in 2016, chatbot design incorporated the first wave of artificial data technology. Through social media platforms such as Facebook, developers can build chatbots for their trademarks or services. (Gobiet, 2024)

Businesses are increasingly moving online to meet customers' needs, so customer service is becoming a more important factor in standing out. However, many existing customers service systems do not provide a seamless customer experience, resulting in frustration when customers have unresolved issues or repeat requests (Holdsworth, 2024). Customer support systems that combine AI technology with human support, such as AI-powered chatbots that integrate ticketing, could significantly improve how businesses interact with customers.

This project created an AI customer support chatbot. We will use this chatbot to efficiently handle real-time customer enquiries and create and resolve issues for human agents. The chatbot provides real-time support for contextual handouts. A Businesses seeking to enhance their customer support operations can benefit from personalized service experiences. Unlike other chatbots, which usually work alone, this system will include a ticketing system, and it keeps chat history and context for human agents. This will guarantee the prompt and consistent handling of customer issues.

The chatbot will utilize complex tools and technologies, including natural language processing (NLP) frameworks and machine learning algorithms to deliver specific and intelligent responses. Self-learning capabilities will enable the chatbot to enhance its performance. Furthermore, enterprises will obtain an analytics dashboard that offers insights into customer interactions and support trends, facilitating informed decision-making.

This project will provide a “RESTful” API with a user-friendly interface that is accessible across multiple platforms such as websites, mobile apps, etc. The client will have the ability to seamlessly integrate the chatbot into their platform, primarily the website, without the need for technical expertise. To ensure proper training, customization, and alignment with specific business needs, a client must first provide a specific dataset when integrating the chatbot with their platform.

Additionally, this project holds significant value for businesses and their customers. As a final stage, implementing such a system will help businesses stay competitive and improve the quality of their services.

Literature Review:

Artificial Intelligence (AI) has revolutionized the customer service process and now companies are able to work more efficiently, cheaper, and deliver better customer experience. AI-based chatbots are the new phenomenon that is making daily customer journey automation and 24 hours availability for instant solution. These are the systems which are part and parcel of customer engagement and whose adoption is a requirement for any business looking to stay competitive in digital economy.

The chatbots that were initially a rule-based model have seen a lot of development in the years. Early chatbots, such as ELIZA in the 1960s, used crude pattern-matching to re-enact conversations. But they didn't have the capacity to make sense of context or answer tricky questions (Weizenbaum, 1966). This disconnect has been filled with AI and NLP based modern chatbots, which make good use of advanced algorithms to process and interpret input from the user.

Now chatbots can be able to take lessons from this previous conversation and make improvements with machine learning. AI chatbots such as OpenAI's GPT versions or IBM Watson Assistant take the language and generate human responses, and offer contextually relevant replies with deep learning algorithms.Considering all of these, chatbots are now the necessary instrument to automate customer service in retail, banking, healthcare and telecom industry (Nguyen, 2020).

Chatbots are used for customer service questions, common issues and also for scheduling an appointment, suggesting products and tracking orders. (Salesforce, 2022) says that AI chatbots are now used in 23% of all customer service requests in the world, so the usage rate is increasing. Companies like Amazon, Uber, Starbucks etc., have already started using chatbots to cut down on the support time thereby providing customers with better customer service.

What is perhaps the biggest advantage of the chatbots is that they are 24/7 allowing you to do away with the need for an agent during off hours. Chatbots can even handle several requests at once, so they are a less expensive solution to resolve mass volumes of customer requests (Zendesk, 2022). Such features all made chatbots core components of the customer service departments.

All the positives but chatbots have negatives too. The main issue is that they can't process complex or fuzzy questions that human input is needed to resolve. Rule-driven chatbots in particular can't scale with the changing customer expectations, and thus create a lot of frustration and unhappiness (Smith, 2021).

Security and privacy is another major problem with chatbots. Large-scale breaches like the one Robinhood suffered in 2020 have brought to light the weaknesses of chatbots.
To keep the user's confidence high and protect personal data, it is very important to adhere to data protection legislations like GDPR in the UK (Abdel-Qader, 2020).

A workaround for chatbot limitations is to use ticketing platforms to push out pending tickets to the agents. Integrations with ticketing systems record customer context and history so human agents can work through issues without having to go through customers again. It's a combination of automation with the reliability of human-powered solutions to increase resolution rates and customer satisfaction (Clymo, 2021).

There are also companies like Zendesk and Freshdesk that have used chatbot-ticketing integrations successfully to streamline the support process. These systems also give the information via analytics dashboards to understand what businesses commonly experience pain points and how they can best support them (Nguyen, 2020).

In the UK, AI chatbots have swept across various sectors. This process was driven by COVID-19 which sped up the adoption of digital solutions, so companies were using chatbots to help stay in touch and fulfil customer needs. According to (TechUK, 2022), 19% of UK companies use AI chatbots in customer service, confirming their rise in digital economy significance.

UK companies – especially in the areas of finance, healthcare and online retail – are using chatbots to deliver individualised, efficient assistance. Banking companies, such as Lloyds and HSBC, for example, have already used chatbots to manage accounts, detect fraud, and apply for loans. In the same way, retailers like ASOS and Tesco use chatbots to deal with product queries and enable online shopping (Times, 2023).

The Advantages of AI Chatbots With intergraded ticketing mechanism:
AI chatbots integration with ticketing system comes with several advantages for both the company and the user:

1. **Increased Efficiency**: Chatbots automate common tasks so human agents have more time to work on difficult problems. This means query execution is faster and it's efficient.

2. **Personalised Assistance**: Chatbots can collect information from users to give personalized advice and solutions to the customer in an optimal customer experience.
3. **Effortless Escalation**: Integrated ticketing systems will escalate the tickets to human agents in full context, so customers don't need to do this over and over again.
4. **Price Effectiveness**: Since all the Customer Support activities are automated, there's no huge requirement of giant human capital anymore which means companies save money.
5. **Scalability**: Chatbots can answer hundreds of queries in one go, so they're an ideal solution for startups or seasonal companies.

AI Customer Assistant Chatbot — Ticketing chatbot to replace the customer support systems. With the help of NLP and ML models, the chatbot will react in the right manner and with the right context for the best customer experience. Here are some of the capabilities of the solution:

1. **Auto-Learn**: The bot will learn from answered tickets and make corrections at the right time, instead of repeating to call agent.
2. **Analytics Dashboard**: Companies will have access to data about customer journeys for better decisions and support.
3. **Multiple Channel Support**: The chatbot will be available on multiple channels (website, mobile apps) with regular support.
4. **Data Privacy**: Data will be kept private and secured by the system, which comply with GDPR.

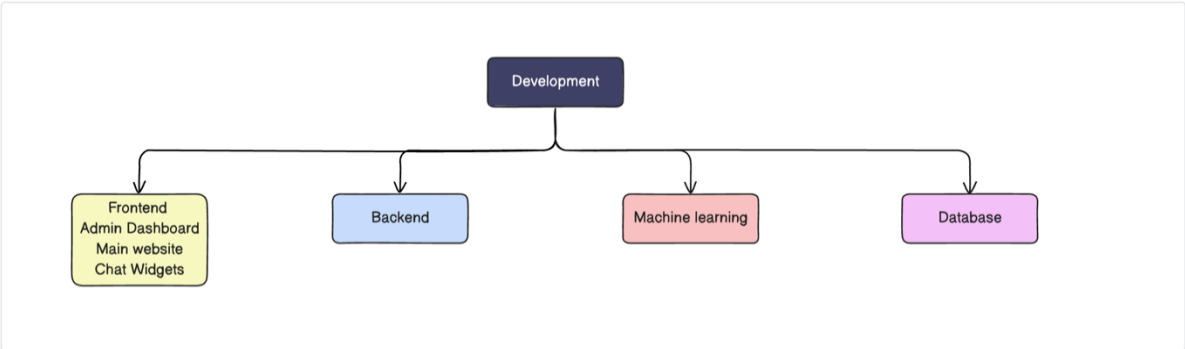
Conclusion

In the literature, we read about the customer service's potential to be transformed by AI chatbots and how they can be more efficient, cost-effective and more customer centric. Security risks and lack of flexibility still loom, but ticketing systems are a viable option to provide an intermediate solution between automation and staff. The AI Customer Assistant Chatbot with ticketing feature we're looking at builds upon all of these and is scalable, adaptable, and effective for UK businesses and beyond.

Technical Review:

The AI chatbot can be developed in a few ways, and after researching, I have divided the project into four main steps with tools for each one.

Figure 1



As I considered how to develop the AI chatbot, there were numerous machine learning frameworks that came to my mind. Such as PyTorch, Keras, Scikit-Learn and TensaFlow among them I decided to use **TensaFlow,** and it is more comfortable for me and TensorFlow is the ML framework of choice because it is very scalable, extensible and production ready. It's deployable in the cloud, on mobile devices (through TensorFlow Lite) and browsers (through TensorFlow.js). TensorFlow's rich set of pre-trained models and compatibility with components such as Keras makes building more advanced models easy, while distributed training makes big-scale projects a snap. As well, TensorFlow comes with a lot of powerful production resources such as TensorFlow Serving for inference in real-time and TensorBoard for monitoring the performance, so it is dependable for research and enterprise work system makes use of TensorFlow.

I am planning to develop front-end for the admin and the users of AI chatbot. In my mind, there were multiple programming languages and frameworks I could use to build the AI chatbot's frontend ,admin frontend and main website frontend. My frontend development framework of choice is **React/Next.js** (Meta framework) among JavaScript, TypeScript, HTML, and TypeScript. Also, I have a lot of experience working with React/Next.js, which provides a robust framework for developing dynamic, high-performance, and scalable user interfaces. Having previously worked with React/Next.js, I can attest to its excellent developer experience.

During my research on AI chatbots, I came across many backend languages and frameworks, including Python, Java, Go, PHP and Node.js. Ultimately, I have chosen **Node.js** as my backend framework because I have done multiple projects with it. Furthermore, I use the React/Next.js framework as a front-end framework, combining React/Next.js with Node.js together provides

a high level of performance for the user. Furthermore, it has the ability to handle a high level of concurrency and real-time interaction at the same time.
I have decided all the tools based on the performance , my skills , scalability, technical compatibility, easy of development ,security and cost. Further details on other methods I could use can be found below where I'll discuss the different methods in depth to; what they are, pros, cons and to why I did not choose to use them for developing the AI chatbot project.

Machine Learning: TensaFlow

I chose TensaFlow for my AI chatbot project because it provides powerful and flexible framework for building and deploying machine learning models. TensorFlow's extensive library of tools and pre-trained models simplifies the development process, enabling me to focus on the unique features of my chatbot rather than starting from scratch.

Its support for Natural Language Processing (NLP) tasks, such as intent recognition, sentiment analysis, and contextual understanding, makes it ideal for building a chatbot that functions as if it were a real person. TensorFlow allows me to leverage advanced architectures (e.g., BERT) to ensure the chatbot understands user input effectively and responds appropriately.

The scalability of TensorFlow makes it easy to train and deploy my chatbot efficiently, regardless of whether I'm working on a small prototype or scaling up to handle real-world traffic. Because it integrates with TensorFlow.js and TensorFlow Lite, I can deploy my chatbot across multiple platforms, such as web and mobile.

TensorFlow's active community and resources enable me to troubleshoot issues and continuously improve my chatbot by incorporating cutting-edge advancements in AI and machine learning. In this way, TensorFlow is the best choice for creating robust, adaptable, and intelligent chatbots.

Frontend: React/Next.js

React and Next.js provide a robust and efficient framework for building responsive and dynamic user interfaces, which is why I chose them for my AI chatbot project. My chatbot and main web application interface is based on React's component-based architecture, which makes development more efficient and maintainable. In addition, I can build interactive visuals and real-time chat features using this method.

By leveraging Next.js' server-side rendering (SSR) and static site generation (SSG) capabilities, I am able to ensure my chatbot as well as the main website load quickly and performs efficiently even on slower connections. The chatbot's success depends on delivering a seamless, responsive user experience, which is a key priority. I can easily incorporate additional features, such as accessibility enhancements and animations, to make both chatbot and main web application interface engaging and user-friendly thanks to React's extensive ecosystem of libraries and tools. I am able to handle real-time interactions and data updates more efficiently using React's hooks support.

Additionally, I can improve SEO optimization for the chatbot's web application by using Next.js, which will make it easier for users to discover and interact with the chatbot online. A professional and user-centric frontend for my AI chatbot and main website can be created with React and Next.js combined. They provide scalability, performance, and flexibility.

Backend: Node.js

I chose Node.js as the backend platform for my AI chatbot and its main web application because it is lightweight, efficient, and highly scalable, perfect for handling real-time communication. A non-blocking, event-driven architecture allows the chatbot to process multiple concurrent user requests seamlessly, providing users with a fast and responsive experience. Additionally, it allows the web application to deliver dynamic content and handle interactions efficiently, even when the traffic is high.

The Node.js framework excels at integrating with APIs, making connecting the chatbot and the web application with backend services and external machine learning APIs, such as TensorFlow, straightforward. Using this integration, the chatbot can process user queries and generate intelligent responses while the web application can manage user data, ticketing, and other backend operations efficiently. The vast ecosystem of libraries and packages available through npm makes it easy to handle databases, manage user authentication, and implement WebSocket connections for real-time communication, thereby benefiting both the chatbot and web application.

One of the key reasons I chose Node.js is its ability to work seamlessly with JavaScript across the entire stack. It simplifies development and reduces complexity by using a single language for developing the chatbot, the backend logic, and the main web application. As both the chatbot and the web application can be integrated with modern database systems like PostgreSQL and MongoDB, the user experience is enhanced across the platform.

Furthermore, Node.js is well-suited for deploying scalable applications that can handle growing traffic and usage as the application and chatbot grow. Node.js provides a powerful, scalable, and easy-to-use backend for both the chatbot and its main web application, due to its performance and scalability.

Backend: PostgreSQL

Using PostgreSQL as the database for my AI chatbot and its main web application allows me to manage both applications' data needs in a powerful, reliable, and scalable manner. I am able to manage structured and unstructured data efficiently, such as user profiles, chat histories, interaction logs, and ticketing details, thanks to its support for structured and unstructured data. Data management is seamless between the chatbot and the web application as a result of this.

Data can be stored and retrieved dynamically with PostgreSQL's robust querying capabilities and advanced features. While the main web application efficiently manages features like user authentication, admin dashboards, and ticket tracking, this allows the chatbot to deliver personalized and context-aware responses. Ticketing information is securely and reliably handled across the platform because of the platform's ACID compliance.

Furthermore, PostgreSQL's scalability and performance optimization capabilities enable both the chatbot and the web application to handle growing amounts of data and traffic. In addition to advanced indexing and full-text search capabilities, the web application is able to provide users and administrators with efficient search and data retrieval capabilities.

In addition to its active community and extensive documentation, PostgreSQL offers invaluable support for development and troubleshooting. Through its compatibility with modern backend technologies like Node.js, the chatbot, main web application, and database are seamlessly integrated. I chose PostgreSQL for powering the data infrastructure of my AI chatbot and its main web application due to its flexibility, reliability, and scalability.

Deployment:

Frontend: Vercel or Netlify

In order to deploy the frontend of my AI chatbot project, I chose Vercel or Netlify because they offer an easy deployment process combined with advanced features like automatic builds and caching. Platforms like these ensure that updates to the frontend are deployed seamlessly, ensuring that users have access to the latest features without experiencing any downtime. The company's CDN (Content Delivery Network) ensures fast loading times by delivering content from the closest server to the user, improving the overall user experience. The platforms also integrate with Git repositories, enabling continuous deployment and efficient version control, making it even easier to manage the project.

Frontend: Railway

As a backend, I chose Railway because its intuitive interface makes deployment easier and it supports multiple regions as well. By deploying the backend closer to the target audience, latency and response times are reduced. In addition to its scalability, Railway makes a great choice as the chatbot grows in popularity and demand. Furthermore, the backend's streamlined environment variable management and ease of integration with databases and storage services contribute to its reliability and flexibility.

Database: PostgreSQL (Neon)

I chose Neon for hosting the PostgreSQL database because it combines the robustness of PostgreSQL with a serverless architecture, allowing for cost-effective scaling. As the database scales automatically and can be paused/resumed according to activity, Neon is ideal for projects with variable traffic patterns. For managing the chatbot's data needs, the chatbot is compatible with modern backend platforms including Node.js and Railway.

Storage: AWS S3 Bucket

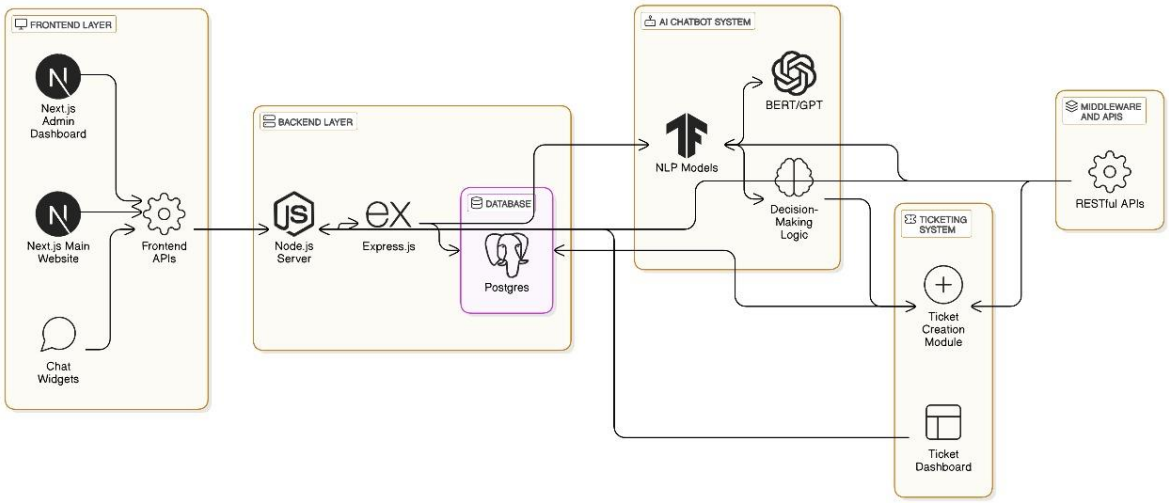
My choice of AWS S3 (Simple Storage Service) enabled me to store datasets and media files securely, scalable, and cost-effectively. S3 is the perfect storage solution for the chatbot, since it supports multiple file formats and offers 5GB of free storage. Featuring lifecycle policies and versioning, it ensures efficient and reliable data storage by being compatible with a wide range of tools and services. Further, S3 integrates with other AWS services such as SageMaker and CloudFront, simplifying file processing and serving.

Machine Learning Model: AWS SageMaker

Using AWS SageMaker, I could easily train, tune, and deploy the machine learning model at scale, because it provides comprehensive tools. TensorFlow integration in SageMaker simplifies the deployment process, which gives me more time to optimize the chatbot's functionality. Having a managed environment makes it reliable for real-time predictions due to its high availability and fault tolerance. S3 integration and dynamic scaling make SageMaker an ideal choice for a production-grade AI chatbot.

With these deployment choices, I ensure a seamless and robust user experience for my AI chatbot while maintaining development efficiency.

Architecture of the project



A structured architecture is used to ensure seamless functionality and interaction between the AI chatbot system and users.Users interact with the chatbot via the Frontend Layer, which includes a website and admin dashboard built with Next.js.Through these interfaces, users can interact with the chatbot and administrators can manage its features. By integrating chat widgets into other websites, a broader audience can access chat services. Through APIs, the frontend and backend exchange data and handle user requests efficiently.

Node.js servers process incoming requests, route them through Express.js, and interact with the database in the Backend Layer. Data stored in PostgreSQL is structured and unstructured, including user profiles, chat histories, and ticket logs. In this way, data consistency and reliability are assured, while personalized and context-aware responses can be retrieved quickly.

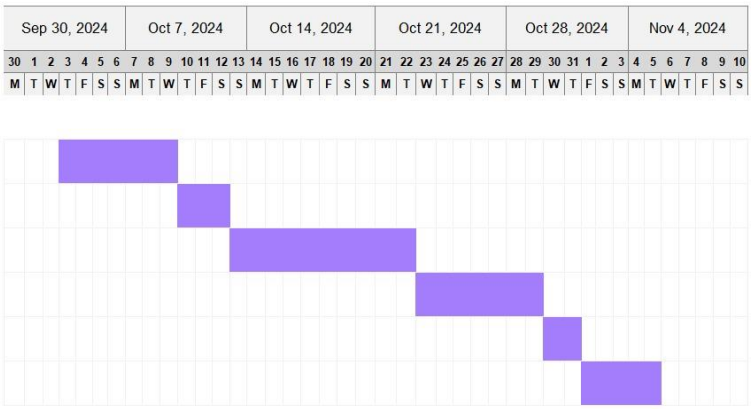
A TensorFlow-based NLP model and advanced frameworks like BERT and GPT power the AI Chatbot System. A model such as this processes inputs from the user, extracts meaning, and generates intelligent responses. When dealing with more complex queries, the chatbot uses decision-making logic to determine the best next step, such as escalating unresolved issues.

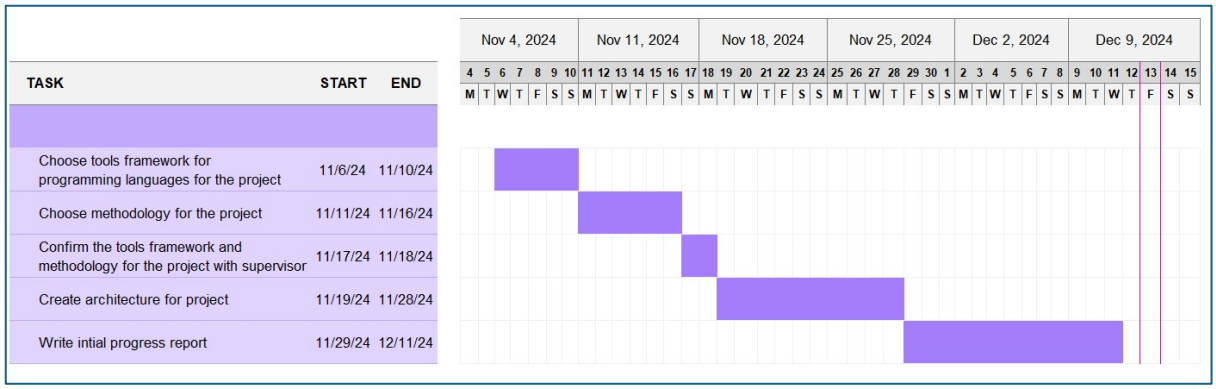
The Ticketing System is used when the chatbot cannot resolve an issue. System's ticket creation module generates a ticket containing relevant context, such as chat history and user details, and assigns it to a human agent. Ticket dashboards enable administrators to efficiently manage and resolve these tickets, ensuring users are assisted as soon as possible.

Finally, middleware and APIs enable seamless data flow and integration with external services. Through this cohesive system, the chatbot provides users with efficient, reliable, and scalable support while optimizing backend operations.

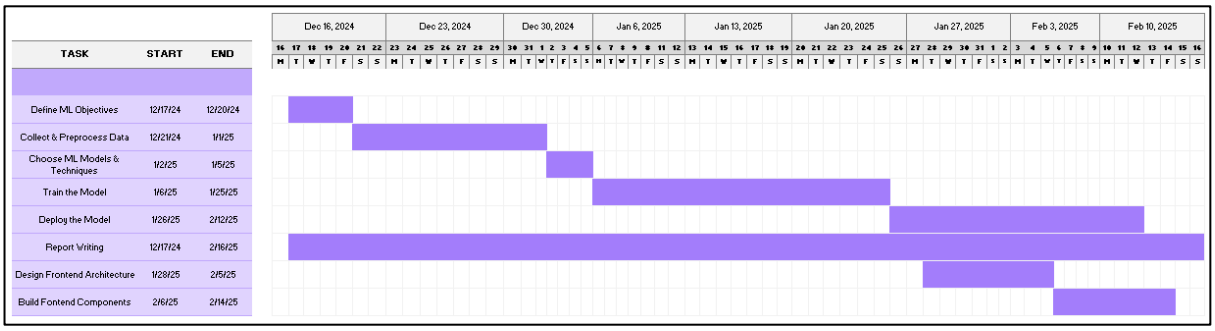
Implementation:

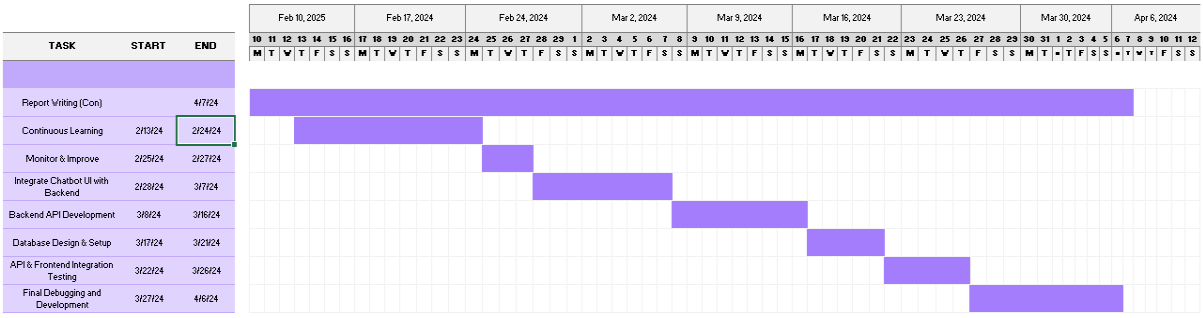
TASK	START	END
Choose the project idea	10/3/24	10/9/24
Choose supervisor	10/10/24	10/12/24
Do the research	10/13/24	10/22/24
Create the project proposal	10/23/24	10/29/24
Confirm the project proposa with the supervisor	10/30/24	10/31/24
Create flow chart for the project	11/1/24	11/5/24





Above Gantt charts explain what tasks have been done during last 10 weeks. During that time, I have gathered the foundation knowledge required for the project. It included research about AI chatbot systems, relevant technologies such as TensorFlow, React, Next.js, and PostgreSQL, as well as deployment platforms like AWS S3 and SageMaker. I chose my supervisor and defined the project scope and objectives. After that I have developed a clear project proposal and outline and confirmed with my supervisor and finally chose methodology and completed initial progress report.





I have chosen Agile and Scrum methodologies to ensure efficient progress and adaptability throughout the development process. I will customize Agile and Scrum to suit my specific workflow since they are usually designed for team-based projects. To maintain momentum and structure, the project will be divided into sprints. Each sprint will focus on a specific task and deliverable.

Sprint 1: Design and Architecture

I will design the overall architecture for the AI chatbot system in this sprint. The frontend, the backend, the database, and the implementation of the chatbot are all part of this process. To outline functionality and access levels for different types of users (e.g., administrators, customers), I will develop wireframes for the web application pages and chatbot. A user story will be developed to capture requirements from a user's perspective. Moreover, I will create a product backlog which describes the tasks and resources necessary for the project. In the event that any gaps are identified from the initial research, further literature review and research will be conducted to justify design decisions.

Sprint 2: Building Frontend

Using Next.js and React, I will implement the designs from Sprint 1 to build the web application's interface and chatbot. In order to accomplish this, user-friendly and responsive UI elements will be created that are aligned with the user stories. It will also include basic functions such as navigation, display of data, and user interaction. To identify usability and functionality issues, an initial test of the UI will be conducted. Any unresolved issues will be documented for later resolution

Sprint 3: Backend Development and Integration

During this sprint, Node.js and Express.js will be used to develop the backend infrastructure. To manage user data, chat histories, and ticket logs, a PostgreSQL database will be set up and connected. To facilitate communication between the front end and the backend, backend APIs will be created. To ensure smooth data flow and functionality, this sprint will also involve initial integration of the frontend and backend

Sprint 4 : Chatbot Integration and Core Functionalities

The trained chatbot model will be integrated into the web application during this sprint. This involves implementing decision-making logic for handling user queries and connecting the chatbot's NLP models (TensorFlow and BERT/GPT).We will also test and develop additional features, such as the creation of tickets and dashboard functionality. Testing will be done on the web application to ensure that the chatbot and other components interact seamlessly. A comprehensive testing process will be conducted to verify the system's functionality and performance.

Sprint 5 : Advanced Features , Final Testing and deployment

My final sprint will focus on adding advanced features, such as personalized user dashboards, analytics displays, and a community forum. The existing features will be refined based on feedback from previous sprints, and extensive testing will be conducted to ensure that all components work together seamlessly. During this sprint, the entire system will also be debugged and deployed to the chosen platforms (e.g., AWS S3, SageMaker, Vercel, or Railway). As part of the final report, any remaining issues will be documented and addressed.

Originality & Use of Generative AI Statement

I understand that to use the work and ideas of others, including AI-generated output, without full acknowledgement, is academic misconduct.

I confirm that this coursework submission is all my own, original work and that all sources, summaries, paraphrases and quotes are fully referenced as required by the LSBU Academic Regulations.

DECLARATION OF AI USE:
I ***DID*** use Generative AI technology in the development, writing, or editing of this assignment. (delete as appropriate)

If you did use Generative AI, please provide detailed responses to the following items:

1. Specify the tools (e.g. ChatGPT, Copilot.) and the purposes for using Generative AI technology in this assignment. Clearly explain how the AI technology assisted in the development, writing, or editing processes.

   I used ChatGPT get some ideas about current AI chatbots in the market. Also, I used to generate most advanced technologies use for AI chatbots in current market. I created ChatGPT customized bot uploading initial progress report guidelines and uploaded my draft take feedback. According to the feedback I have change my technical review part and implementation part. I didn't use anything directly from ChatGPT.

2. Outline the sections or parts of the assignment that were developed, written, or edited with the assistance of Generative AI technology.

   I get feedback from the ChatGPT about the draft and its suggests me to do some changes in technical review part and implementation part.

3. Provide 2-3 examples of prompts you used when using AI tools for this assignment.

   "I'm going to develop AI Customer Assistant Chatbot using integrated ticketing system. Please give me what are the current chatbot use in industry related to my project"

   'This is my initial progress report draft. Please give me feedback to improve my draft"

4. Reflect on how Generative AI technology contributed to the assignment, including its limitations and advantages in the context of the coursework

Generative AI (ChatGPT) helped with this project, offering insight, feedback, and suggestions for better clarity and organization. It was particularly useful for ideation, content planning, and argument refining. But its conclusions were likewise sometimes too broad and needed some fine tuning to fit the breadth and demands of the course. AI provided some helpful guidance, but the work, critique and conclusion were entirely mine, which ensured originality and critical engagement with the content.

By including this statement in my coursework submission, I attest that the information provided in this Originality and use of Generative AI Statement is accurate and complete to the best of my knowledge. I understand that providing false information is a violation of the LSBU Academic Regulations and may result in academic and/or disciplinary consequences.

References:

Marie Gobiet (2024). *The History of Chatbots - From ELIZA to Alexa*. [online] Chatbots and Voice Assistants from Onlim. Available athttps://onlim.com/en/the-history-of-chatbots/.

Holdsworth, J. (2024). *Benefits of Chatbots | IBM*. [online] Ibm.com. Available at:https://www.ibm.com/think/insights/unlocking-the-power-of-chatbots-key-benefits-for-businesses-and-customers.

Giva (2023). *AI Ticketing & 10 Benefits to Customer Support Businesses | Giva*. [online] Giva. Available at:https://www.givainc.com/blog/ai-ticketing/

Weizenbaum, J. (1966) 'ELIZA—A computer program for the study of natural language communication between man and machine', *Communications of the ACM*, 9(1), pp. 36–45.

Nguyen, M. and Rosenbaum, S. (2020) 'Towards understanding fintech and its industry', Journal of Financial Innovation, 12(3), pp. 45–58.

Mohamed, R. and Ali, T. (2019) *The Role of Artificial Intelligence in Financial Services*. BusinessTech Publications.

Salesforce (2022) 'The Future of Customer Service with AI Chatbots'. Available at: https://www.salesforce.com (Accessed: 2 December 2024).

Zendesk (2022) 'The Benefits of AI-Powered Customer Support'. Available at: https://www.zendesk.com (Accessed: 2 December 2024).

Smith, J. (2021) *Customer Experience in the Digital Age*. TechPress Publishing.

Clymo, R. (2021) 'Chatbots and Ticketing Systems: A Hybrid Approach', *Tech Innovations Journal*, 14(5), pp. 23–29.

TechUK (2022) 'AI Chatbots and Digital Transformation in the UK'. Available at: https://www.techuk.org (Accessed: 2 December 2024).

Financial Times (2023) 'AI Chatbots in UK Business: Opportunities and Challenges'. Available at: https://www.ft.com (Accessed: 2 December 2024).

Abdel-Qader, H. (2020) 'Security Risks in Chatbot Systems', *Cybersecurity Today*, 7(2), pp. 56–64

Paul, K. and Rushe, D. (2020) 'Tech Companies Thrive Amidst the COVID-19 Pandemic', *The Guardian*. Available at: https://www.theguardian.com (Accessed: 2 December 2024).

OpenAI (2023) 'Natural Language Processing: Enhancing Customer Conversations'. Available at: https://openai.com (Accessed: 2 December 2024).

freeCodeCamp.org. (2024). *PyTorch vs TensorFlow – Which is Better for Deep Learning Projects?* [online] Available at: https://www.freecodecamp.org/news/pytorch-vs-tensorflow-for-deep-learning-projects/.

Lawson, L. (2023). *Why Viable Uses Next.js and Node.js for AI Applications*. [online] The New Stack. Available at: https://thenewstack.io/why-viable-uses-next-js-and-node-js-for-ai-applications/ [Accessed 13 Dec. 2024].

Acharya, D. (2023). *How Node.js Can Be Used in Making AI-Driven Applications*. [online] Medium. Available at: https://medium.com/@debasisacharya1977/how-node-js-can-be-used-in-making-ai-driven-applications-c31256ef39bb [Accessed 13 Dec. 2024].

Amazon Web Services, Inc. (2024). *Unified data, analytics and AI*. [online] Available at: https://aws.amazon.com/sagemaker/?trk=b6c2fafb-22b1-4a97-a2f7-7e4ab2c7aa28&sc_channel=ps&ef_id=Cj0KCQiA0--6BhCBARIsADYqyL-iBDP7EKvTOqt4HDfvdrKY9FVK9kh_8KX8x8dvlUbZqlmGa0RsSmgaAtUeEALw_wcB:G:s&s_kwcid=AL [Accessed 13 Dec. 2024].

Neon. (2022). *Neon Postgres*. [online] Available at: https://neon.tech.

Railway. (n.d.). *Railway*. [online] Available at: https://railway.app.

Munteanu, A. (2024). *If you're working with AI, you're working with data. From numerical data to videos or images, regardless of your industry or use case, every AI project depends on data in some form. The question is: how can you efficiently store that data and use it when building your models? One answer is PostgreSQL, a proven […]*. [online] Ubuntu. Available at: https://ubuntu.com/blog/postgresql-ai-application [Accessed 13 Dec. 2024].